Life Sciences and
Facility Management

Institute of
Computational Life Sciences

Fourier Transform in 2D

Image and Signal Processing

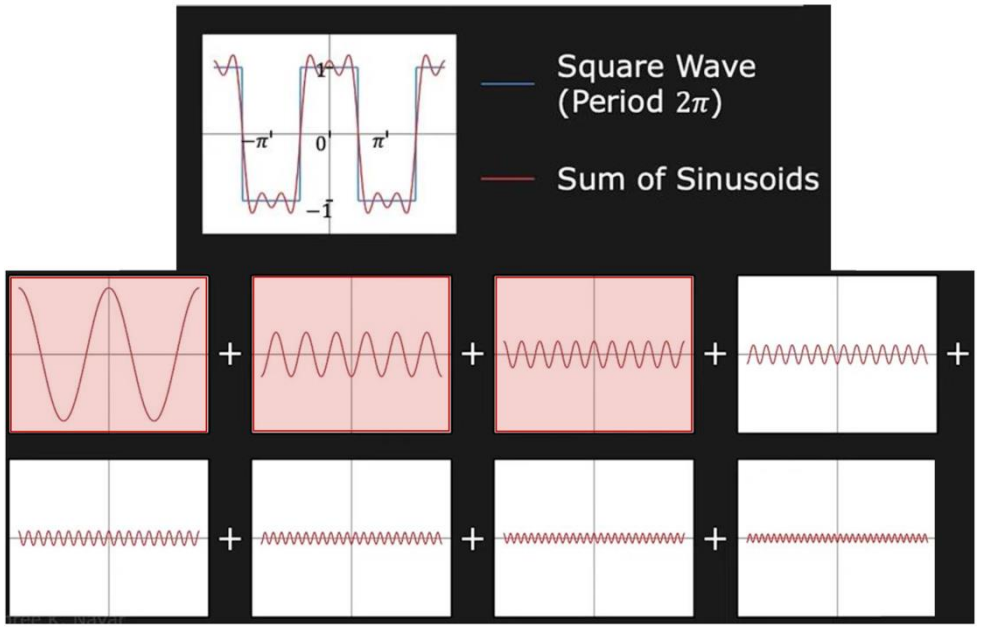
Norman Juchler

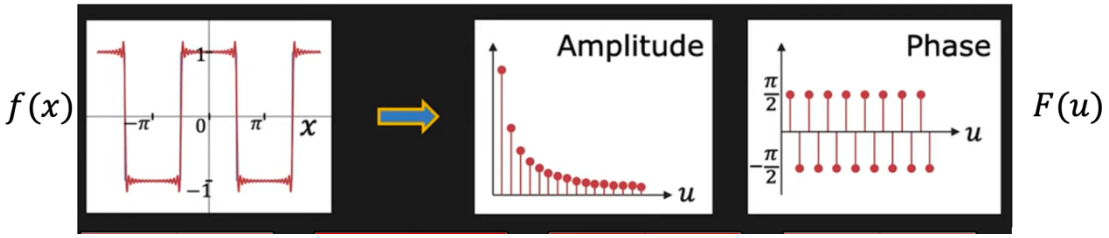
Fourier series / Fourier transforms

Central concept:

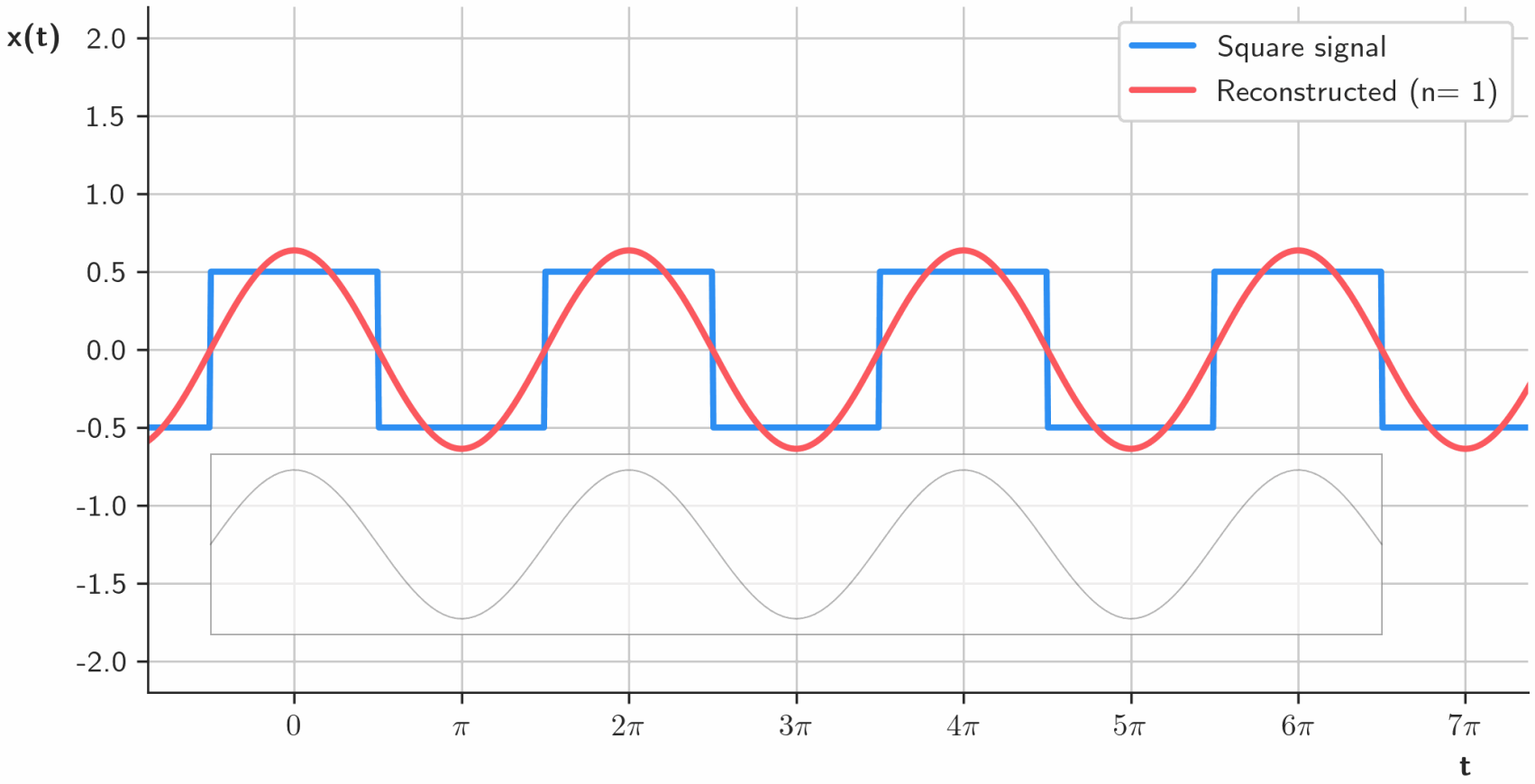
- Each periodic function can be represented by a weighted sum of infinite sine functions of different frequency, amplitude, and phase.

- This central concept can be generalized to non-periodic signals, which leads to the **Fourier transformations**



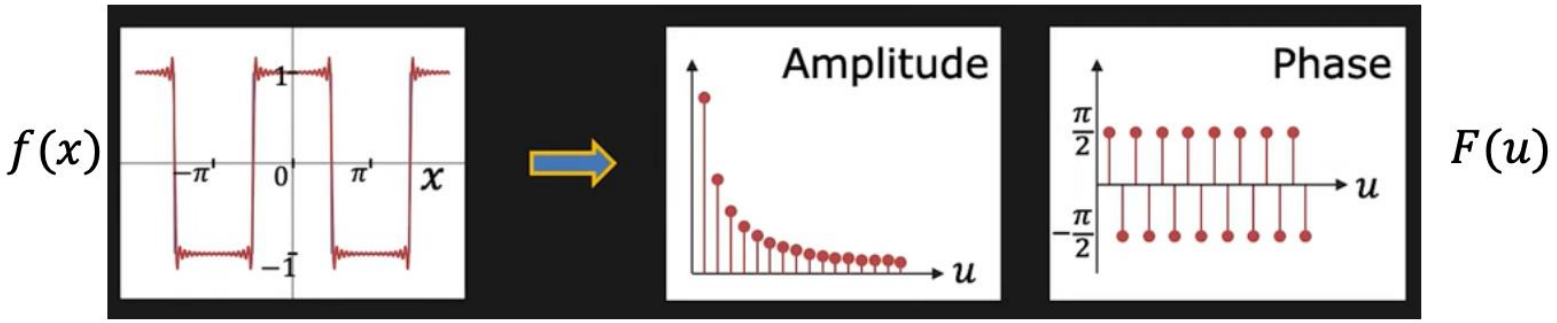


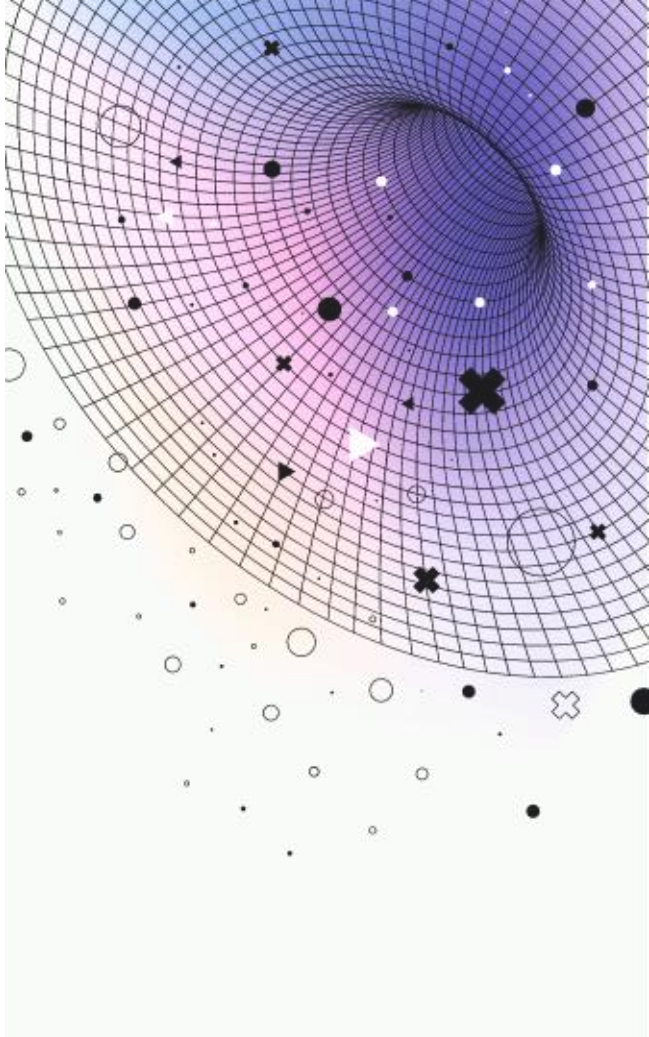
Example: Fourier series of a square signal



Fourier series / Fourier transforms

- With the **Fourier transformation**, we bring a continuous-time signal into the frequency domain
- With the **inverse Fourier transformation**, we bring a frequency domain signal back into the time domain

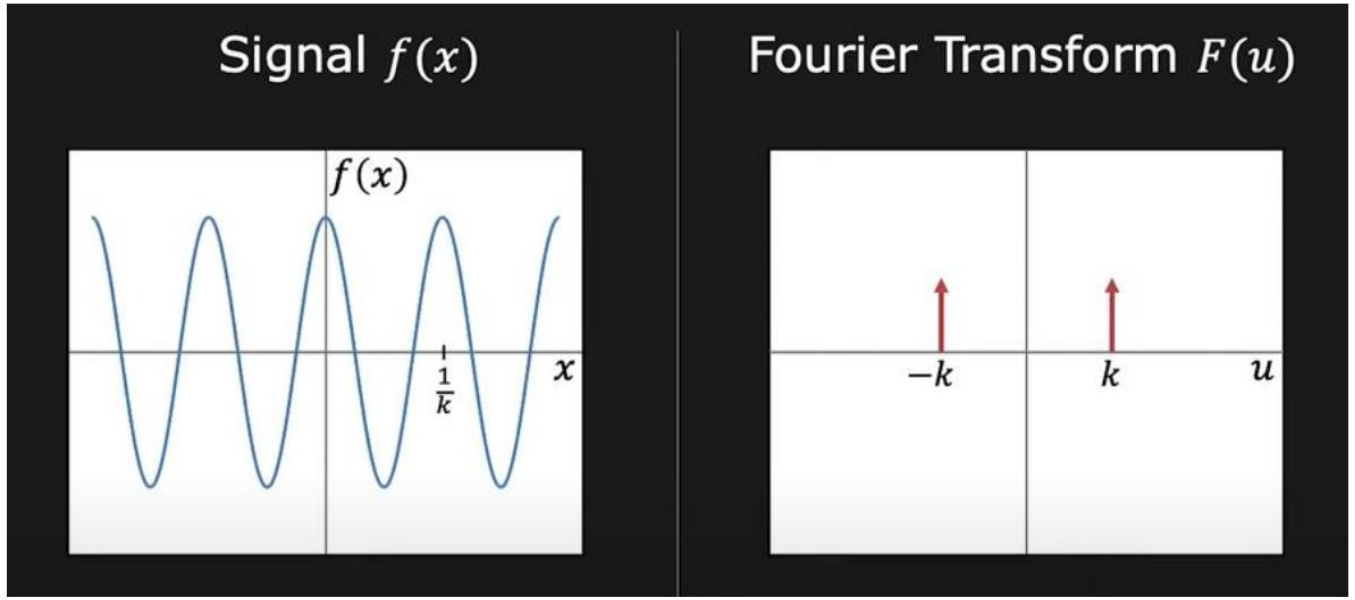




See the Fourier transform as a mathematical wormhole ☺

Fourier series / Fourier transforms

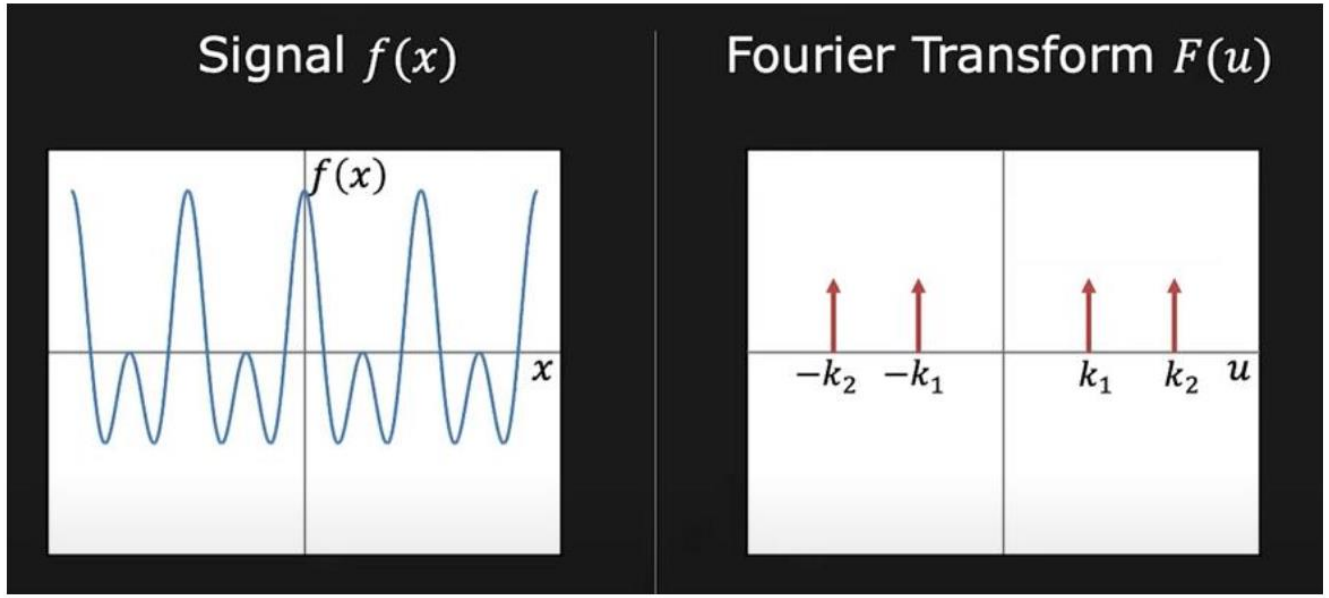
Example



$$f(x) = \cos(2\pi kx) \qquad F(u) = \frac{1}{2}[\delta(u+k) + \delta(u-k)]$$

Fourier series / Fourier transforms

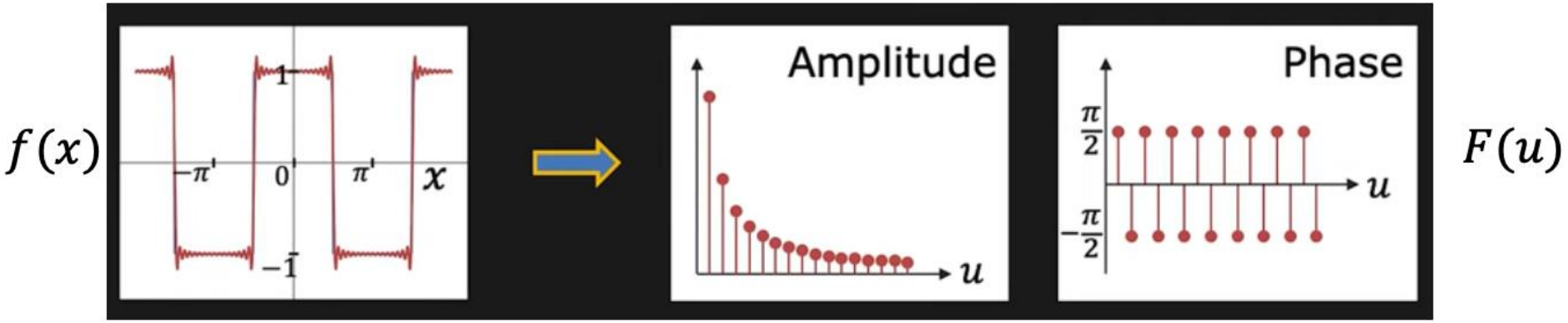
Example



$$f(x) = \cos(2\pi k_1 x) + \cos(2\pi k_2\, x)$$

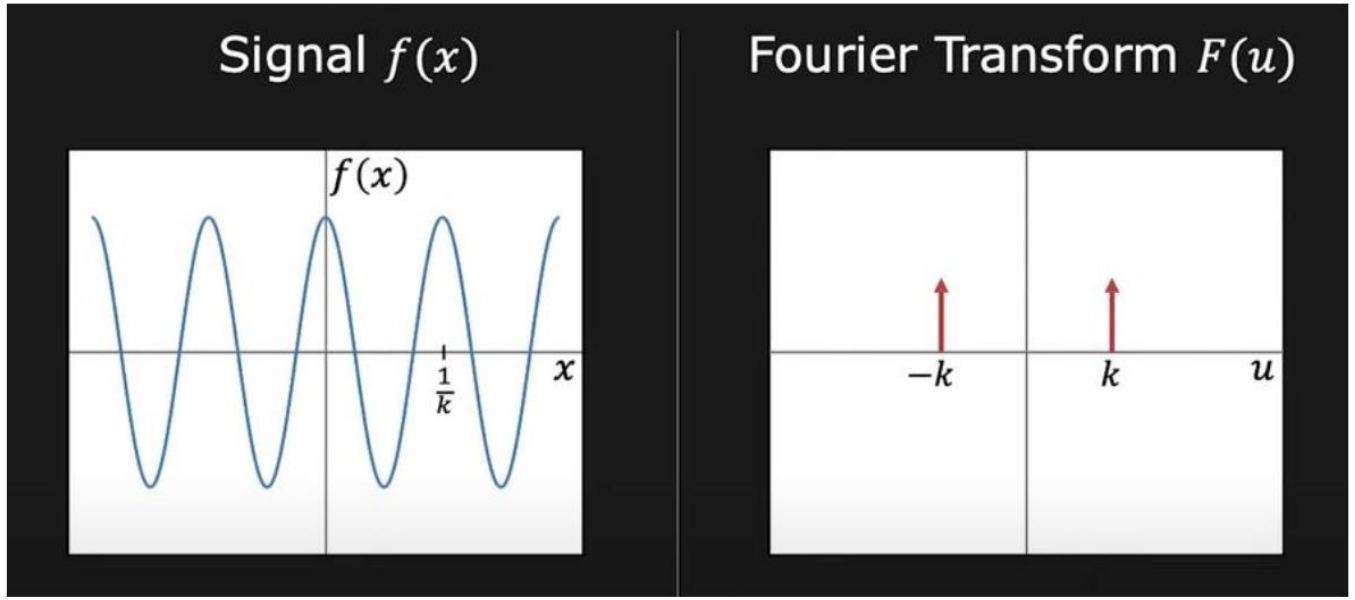
$$F(u) = \frac{1}{2}[\delta(u + k_1) + \delta(u - k_1) + \delta(u + k_2) + \delta(u - k_2)]$$

Fourier series / Fourier transforms



Fourier series / Fourier transforms

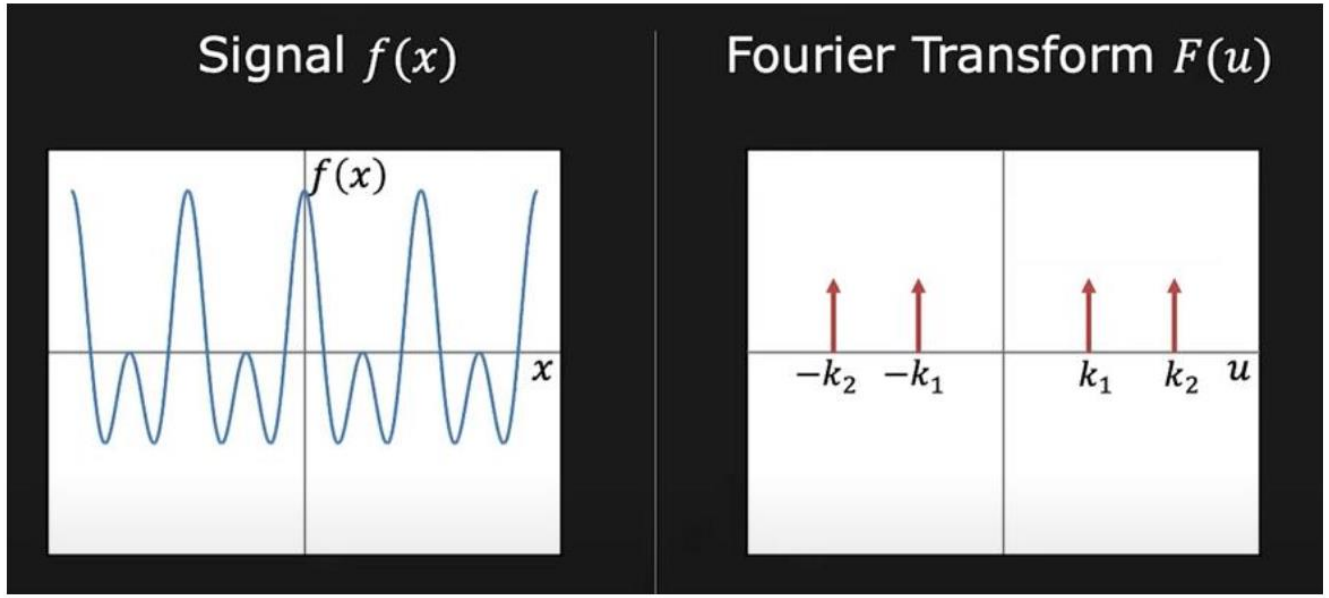
Example



$$f(x) = \cos(2\pi kx) \qquad F(u) = \frac{1}{2}[\delta(u+k) + \delta(u-k)]$$

Fourier series / Fourier transforms

Example



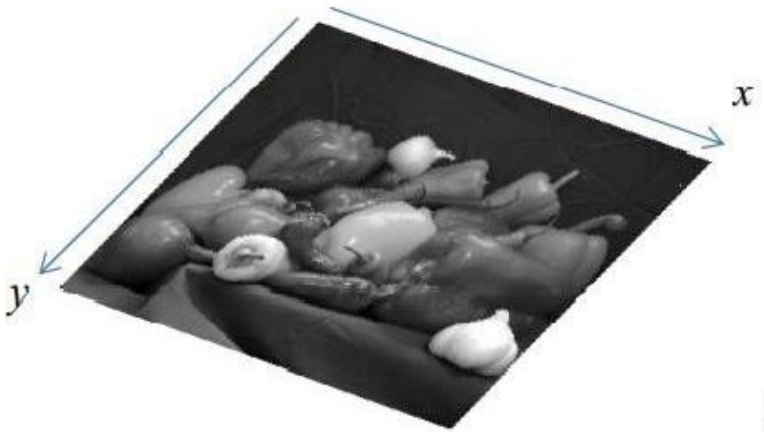
$$f(x) = \cos(2\pi k_1 x) + \cos(2\pi k_2\, x)$$

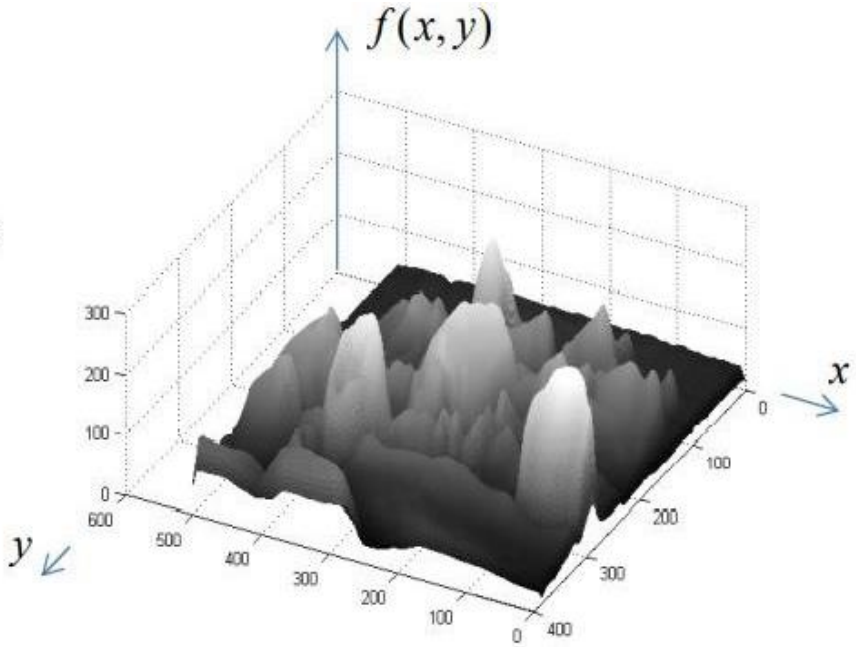
$$F(u) = \frac{1}{2}[\delta(u + k_1) + \delta(u - k_1) + \delta(u + k_2) + \delta(u - k_2)]$$

Do you remember that we can see an image as a picture…?

- A painted grayscale image can be considered as a continuous function over a region in $\Omega \in \mathbb{R}^2$, then at each point (x, y) point we get a gray value g which we can describe by a number $g = f(x, y) \in \mathbb{R}$:

$$f : \Omega \in \mathbb{R}^2 \to \mathbb{R}, \quad (x, y) \to f(x, y)$$





Let us adopt this perspective to apply Fourier methods to images and analyze their frequency content.

Generalization to 2D

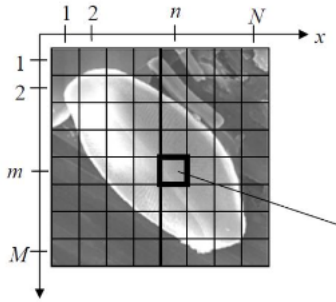
- The Fourier transform – and its discrete variants – can be naturally extended to two dimensions.

Discrete Fourier Transform (DFT)

$$F[u,v] = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f[x,y] e^{-\frac{i2\pi(ux)}{M}} e^{-\frac{i2\pi(vy)}{N}}$$

Inverse Discrete Fourier Transform (IDFT)

$$f[x,y] = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F[u,v] e^{\frac{i2\pi(ux)}{M}} e^{\frac{i2\pi(vy)}{N}}$$



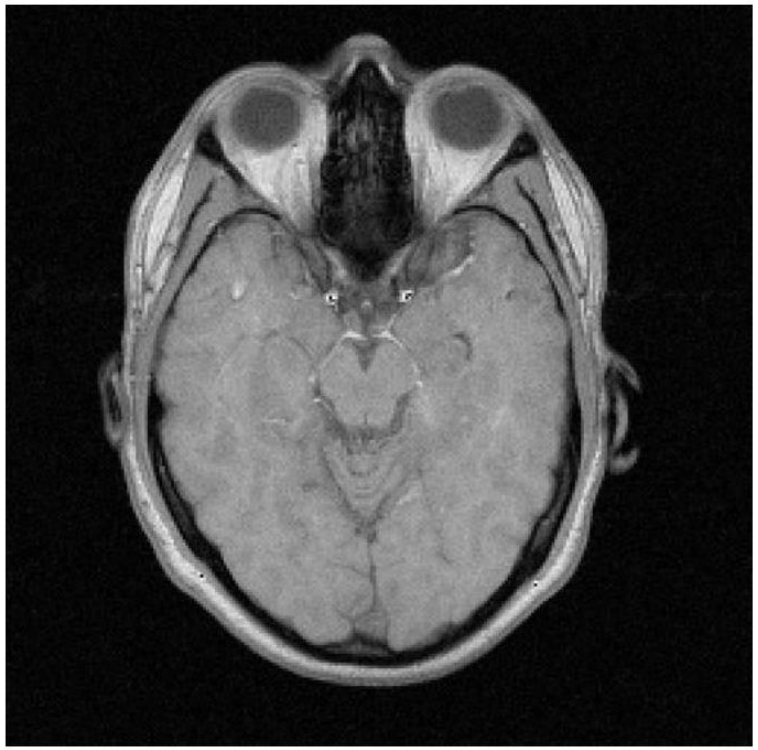
Row Index $1 \leq x \leq M$
Column Index$1 \leq y \leq N$

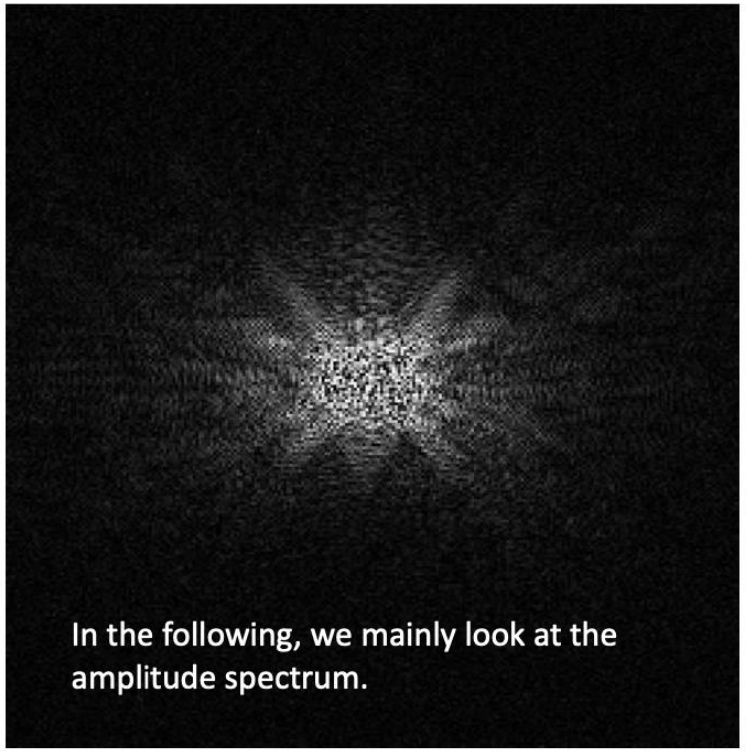
Pixel (x, y)

x, y: Image coordinates
u, v: Frequencies
$i = \sqrt{-1}$

This is called a **linear, bijective** transformation → **no image information is lost.**

Frequency domain representation of an image



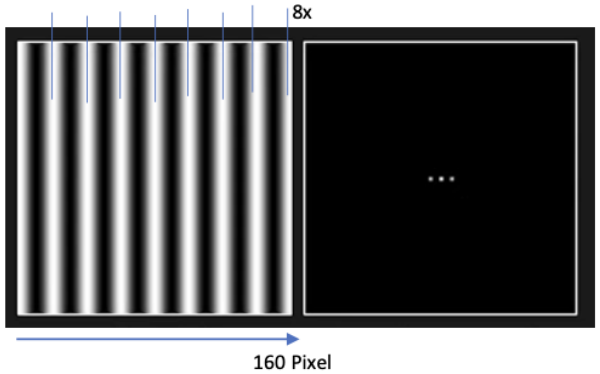


Frequency domain representation of an image

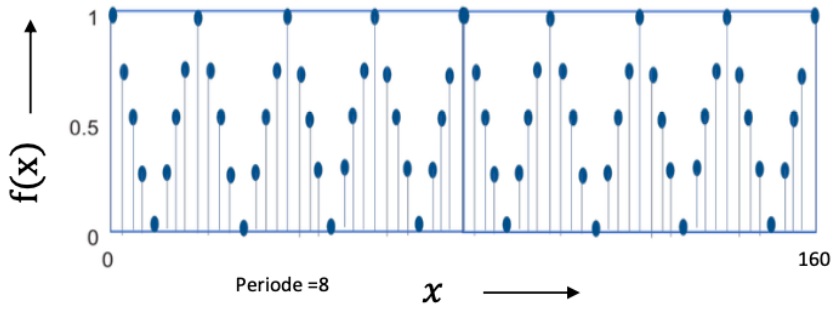
Examples:

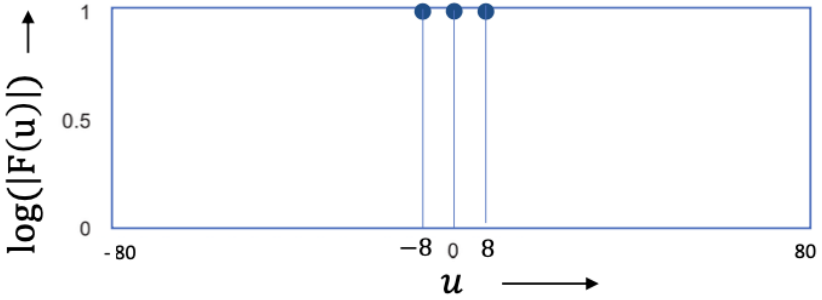
- $f(x, y)$ is a cos function in the x-direction
- We look at the logarithm of the amplitude spectrum:

$f(x, y)$



$\log(|F(u, v)|)$





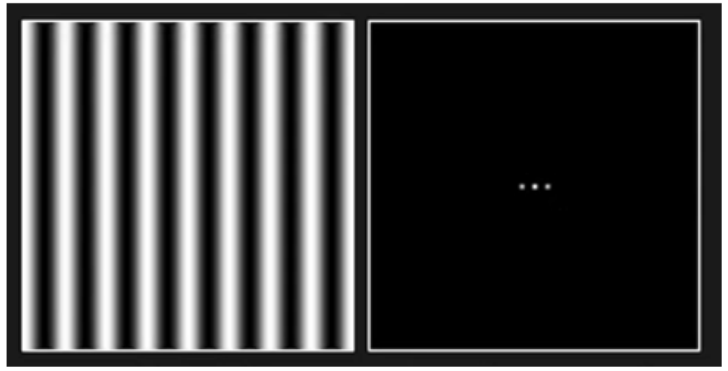
Frequency domain representation of an image

Examples:

- $f(x, y)$ and $g(x, y)$ are cos functions in the x-direction
- We look at the logarithm of the amplitude spectrum:

$f(x, y)$

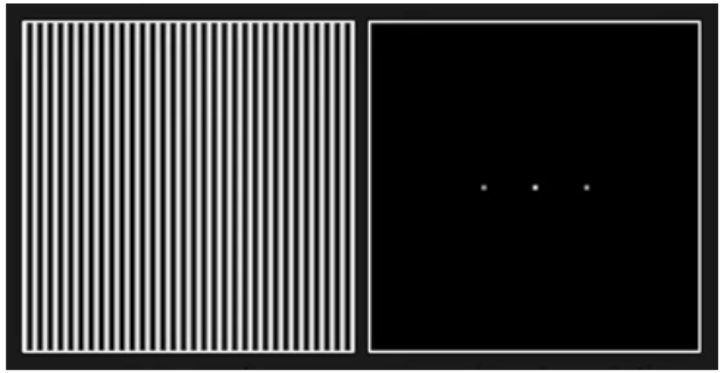
Periode =8



$\log(|F(u, v)|)$

$g(x, y)$

Periode =32

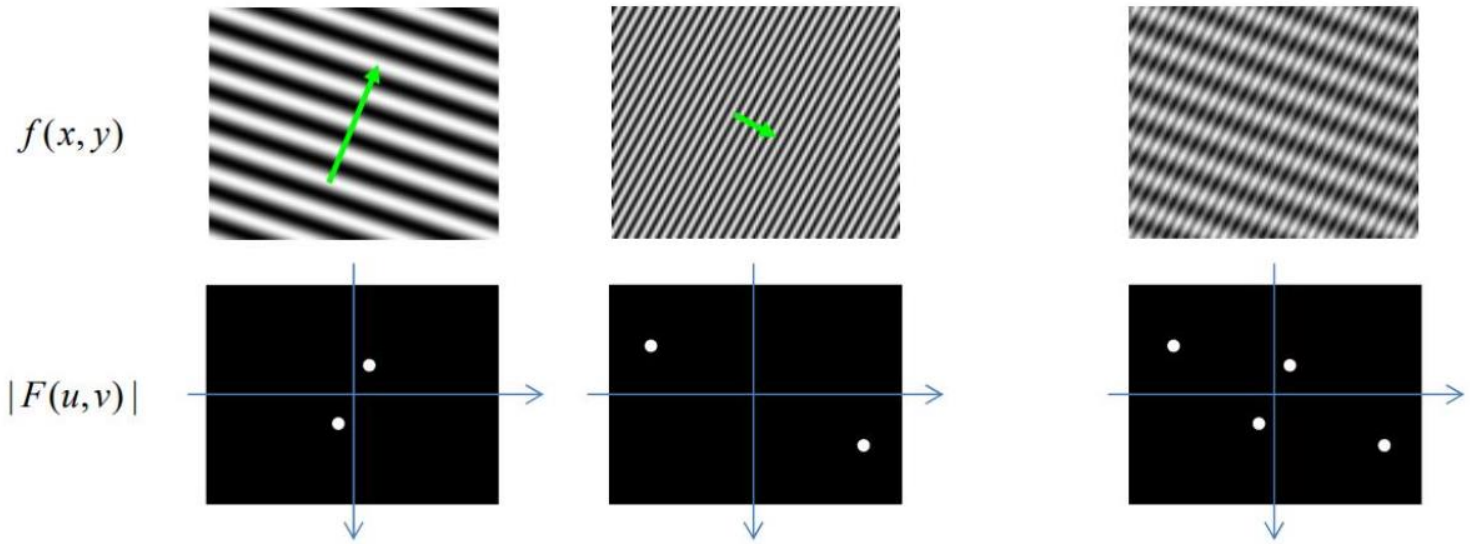


$\log(|G(u, v)|)$

Frequency domain representation of an image

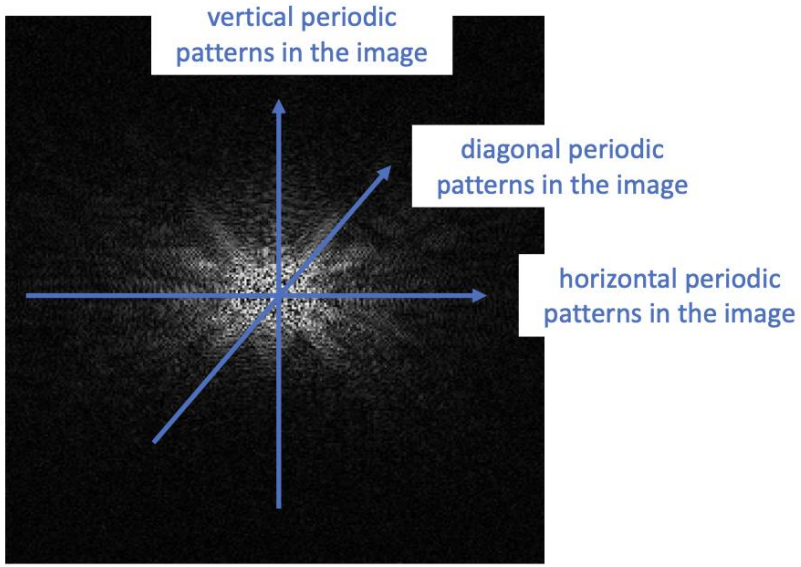
Interpretation of the Fourier spectrum

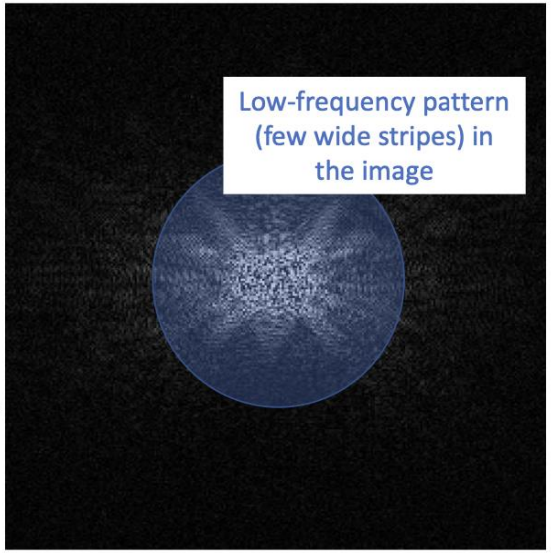
A wave in the image f(x,y) is always linked to a point pair in the amplitude spectrum |F(u,v)|. Thus, the distribution of the significant frequencies in the image can be recognized in the amplitude spectrum.

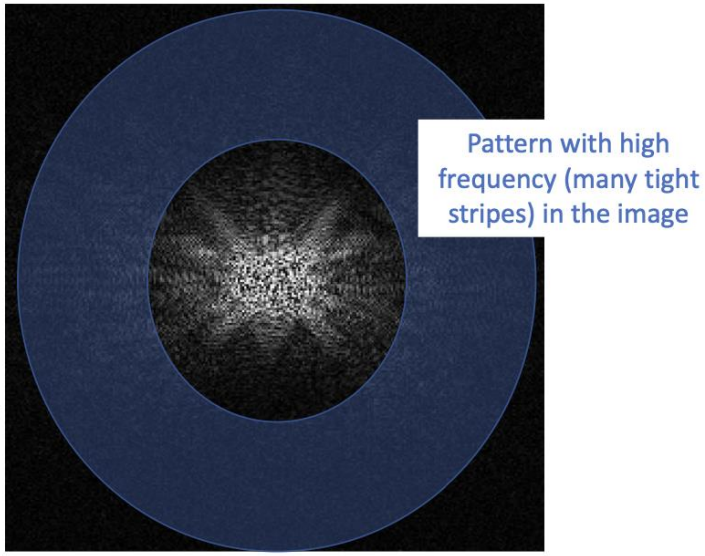


Interpretation of the Fourier spectrum

In the Frequency domain representation, an image is interpretated as sum of harmonic functions with frequency u and v. From the amplitude spectrum conclusions on the frequencies present in the image can be drawn.







The discrete Fourier transform (1D) – a naive implementation:

```
1  def dft(x):
2      '''Compute the Discrete Fourier Transform of an input signal x of N samples.'''
3
4      N = len(x)
5      X = np.zeros(N, dtype=np.complex)
6
7      # Compute each X[m]
8      for m in range(N):
9
10         # Compute similarity between x and the m'th basis
11         for n in range(N):
12             X[m] = X[m] + x[n] * np.exp(-2j * np.pi * m * n / N)
13
14     return X
```

Question: How long does it take to compute the DFT for a signal x?

Answer: Line 12 needs to be calculated N^2 times, or using the big-O notation: The complexity of this DFT is $O(N^2)$

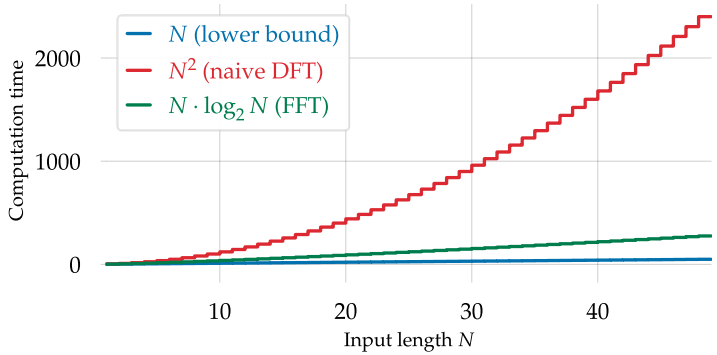
Code sample taken from Brian McFee / Digital Signals Theory

The Fast Fourier Transform (FFT)

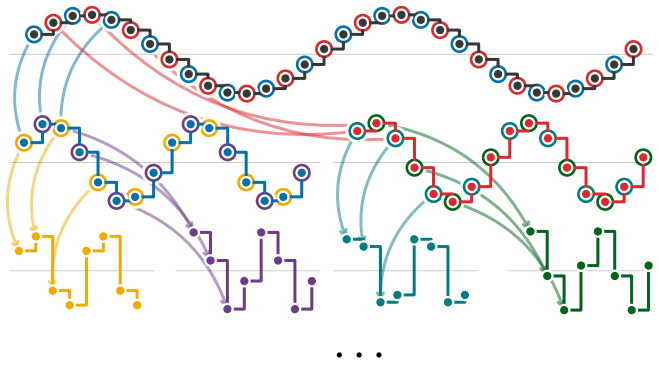
- The FFT is an efficient algorithm to compute the DFT of a discrete signal

- History:
  - Some ideas of FFT can be traced back to C.F. Gauss (1805).
  - The algorithm was first published in 1965 by J. Cooley and J. Tukey.
  - The most popular version is called radix-2 Cooley-Tukey algorithm.

- Key idea: **Divide and conquer**
  - Exploit the symmetry properties of the DFT and divide the computation into smaller sub-problems recursively.
  - Instead of solving the DFT for $N = N_1 N_2$, it solves the DFT in terms of for N_1 smaller problems of size N_2.
  - The complexity of the algorithm is O(N log N), compared to O(N^2) for a naive implementation of DFT



The increase of the amount of work (that is, the number of computations) increases with the problem size

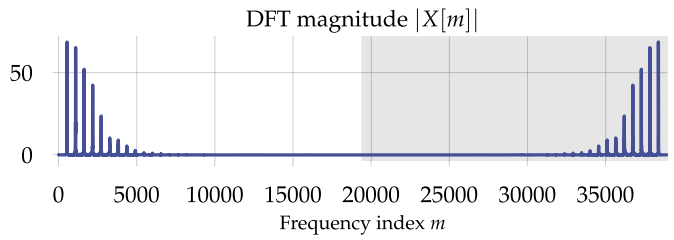


A discrete signal of N=32 samples (top row) is divided into its even and odd samples (middle-left and middle-right).

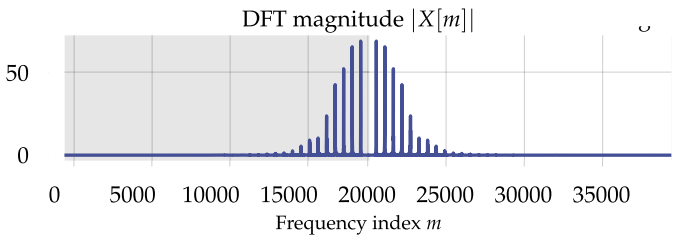
Illustrations: Brian McFee / Digital Signals Theory

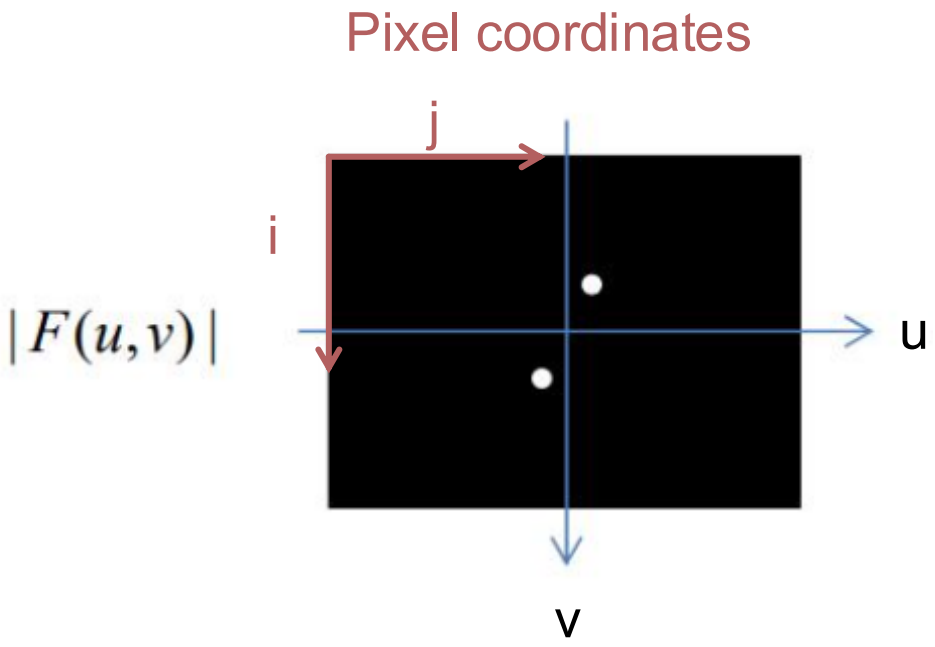
The Fast Fourier Transform (FFT)

- The package scipy.fft offers fast implementations of various FFT algorithms
- See this tutorial for an overview how to use FFT.
- The main functions:
  - `fft(), fft2(), fftn():` Compute the FFT on 1D, 2D, and nD input.
  - `ifft(), ifft2(), ifftn():` Compute the inverse DFT in 1D, 2D or nD
  - `rfft(), rfft2(), rfftn():` Compute the FFT on real input, faster than the fft*() functions.
  - `irfft(), irfft2(), irfftn():` Compute the inverse real DFT in 1D, 2D or nD
  - `fftshift():` fft*() and fftfreq() return all components at positive frequencies, followed by components at negative frequencies. This function swaps the two halves so that the zero-frequency component is in the center: [0, 1, 2, -3, -2, -1] → [-3, -2, -1, 0, 1, 2, 3].
  - `fftfreq(), rfftfreq():` Get the frequencies of each bin returned by the FFT functions, such as the X-axis.



`fftshift()` →



DFT spectra as images

- Recall:
  - $F(u,v)$ in general is complex:
    $F(u,v) \in \mathbb{C} \quad \leftrightarrow \quad F(u,v) = |F(u,v)| \cdot e^{i\varphi}$
  - $|F(u,v)|$ refers to the amplitude, and
    $\varphi = \angle(F(u,v))$ to the phase
- We can represent amplitude (or phase) spectra of DFTs as images (`img = |F(u,v)|`)
- The spectral images have the same dimensions as the input images!
- **Note**: The spectra include negative frequencies! This means that:
  $|F(0,0)|$ = `img[h//2, w//2]`